Jira Task 04 – Inter Service Communication

Step 1

1. REST Communication

REST (Representational State Transfer) is a way for services to communicate over HTTP.
In microservices, one service can call another service using REST APIs.

Example:
Order Service → User Service → GET /users/101

2. HTTP

HTTP (Hypertext Transfer Protocol) is a communication protocol used to send requests and responses between services.

Common HTTP methods:

- **GET** – Retrieve data
- **POST** – Create data
- **PUT** – Update data
- **DELETE** – Delete data

3. JSON

JSON (JavaScript Object Notation) is a lightweight format commonly used to exchange data between microservices.

Example:

```
{
  "userId": 101,
  "name": "Raju",
  "email": "raju@example.com"
}
```

4. Synchronous Communication

In synchronous communication, the calling service **waits for a response** from the other service.

Example:

Order Service → User Service → Response → Order Service

If User Service is unavailable, Order Service may have to wait or receive an error.

5. Asynchronous Communication

In asynchronous communication, the calling service **does not wait for an immediate response**. Communication can happen through a message broker or event system.

Example:

Order Service → Message Queue → Notification Service

This allows services to work more independently.

Step 2

1. **User Service**
    a. Handles user-related operations.
    b. Example: Get user details.
2. **Order Service**
    a. Handles order-related operations.
    b. Example: create and retrieve orders.

For the task, create these as two separate Java/Spring Boot projects:

Microservices

│

├── User Service

│

└── Order Service

Create **User Service** and **Order Service** as separate services.

Step 3 — Create User API

GET /api/users/{id}

It receives a user ID and returns that user's details.

Example:

GET http://localhost:8081/api/users/101

Response:

```
{
  "id": 101,
  "name": "Raju",
  "email": "raju@example.com"
}
```

In the Eclipse project

Create a controller called **`UserController`**:

```
@RestController
@RequestMapping("/api/users")
public class UserController {

    @GetMapping("/{id}")
```

```
    public String getUser(@PathVariable int id) {
        return "User ID: " + id;
    }
}
```

Step is completed when your User Service has:

`GET /api/users/{id}`

Step 4 — Create Order API

We need to create a **GET API** in the **Order Service**:

GET /api/orders/{id}

It receives an **order ID** and returns the order details.

Example:

GET http://localhost:8082/api/orders/101

Example response:

```
{
  "id": 101,
  "userId": 1,
  "product": "Laptop",
  "quantity": 1
}
```

Java code for `OrderController`

```
@RestController

@RequestMapping("/api/orders")

public class OrderController {


   @GetMapping("/{id}")

   public String getOrder(@PathVariable int id) {

      return "Order ID: " + id;

   }

}
```

The Step is completed when your Order Service has:

`GET /api/orders/{id}`

Step 5 — Make Order Service Call User Service

The goal is to make the **Order Service communicate with the User Service using REST/HTTP**.

For example:

Order Service

|

| GET /api/users/1

↓

User Service

|

| User details

↓

Order Service

Simple Java implementation

In the **Order Service**, use `RestTemplate` to call the User Service:

```
RestTemplate restTemplate = new RestTemplate();


String url = "http://localhost:8081/api/users/" + userId;
```

String userResponse =

restTemplate.getForObject(url, String.class);

Assuming:

- **User Service:** `localhost:8081`
- **Order Service:** `localhost:8082`

So when the Order Service needs user information, it sends:

GET http://localhost:8081/api/users/1

and receives the user's response.

Step 6 — Implement

We need to implement **four things** for communication between the **Order Service** and **User Service**:

1. **Request**
   Order Service sends a request to User Service.
   GET http://localhost:8081/api/users/1

2. **Response**
   User Service returns the user details.

```
{
 "id": 1,
 "name": "Raju",
 "email": "raju@example.com"
}
```

3. **Timeout**
   If User Service takes too long to respond, the Order Service should stop waiting and handle the timeout.

User Service not responding
↓
Timeout
↓
Order Service handles it

4. **Error Response**
   If the User Service is unavailable or returns an error, the Order Service should handle it properly instead of crashing.

User Service unavailable
↓
Error response
↓
"Unable to fetch user details"

Step 7 — Test

You need to test the communication between Order Service and User Service in three cases:

1. User exists

Send a request for a valid user ID.

GET /api/orders/101

Result: Order Service successfully gets the user details from User Service.

2. User does not exist

Send a request with an invalid/non-existing user ID.

GET /api/users/999

Result: User Service returns **404 Not Found** or an appropriate error.

3. User Service unavailable

Stop the User Service and then call the Order Service.

Result: Order Service handles the connection failure/timeout gracefully and returns an appropriate error instead of crashing.

Three scenarios:

- Valid user → successful response.
- Invalid/non-existing user → 404/error response.
- User Service unavailable → timeout/connection error is handled properly.

Step 8 — Document Synchronous vs Asynchronous Communication

Synchronous	Asynchronous
Sender waits for a response.	Sender does not wait for an immediate response.
Communication happens directly between services.	Communication can happen through a message broker/queue.
Easier to implement and understand.	More scalable and loosely coupled.
If the receiving service is unavailable, the request may fail or time out.	The message can remain in the queue until the receiving service is available.
Example: REST API call.	Example: Kafka/RabbitMQ message.

Example for this project:

Synchronous:

Order Service

↓ REST Request

User Service

↓ Response

Order Service

Asynchronous:

Order Service

↓ Message

Message Queue

↓

User/Notification Service

Conclusion:

Synchronous communication is suitable when the Order Service needs an immediate response from the User Service.

Asynchronous communication is useful when services should work independently without waiting for an immediate response.